SE LAB 7: Fuzzing for Input Validation Bugs

Field	Details
Name	Samiksha BN
SRN	PES1UG24CS413
Course	Software Engineering, Semester 5
Application under test	daycare_fees.py (processor.py)
Test file	test_daycare_fees.py (test_processor.py)
Tools	Python 3.12, Hypothesis, pytest
Repository	https://github.com/SamikshaBN/SE-Labs-PES1UG24CS413 (folder Lab7)
Forked repo	https://github.com/SamikshaBN/L6_L7_48_daycare_fees

3. Step 2: Execution of the Original (Buggy) Application

Command: python daycare_fees.py (run on the original code, before any fix)

3.1 Normal input

Inputs used: toddler:25, then 20,25, then 3000|250. The average of 20 and 25 is 22.5, but the program prints 22 because of floor division.

```
(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler:25
Age Group and Hours: ('Toddler', 25.0)

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,25
Average Hours: 22

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 3000|250
Payable Fee: 3290.0
```

3.2 Malformed input

Input used: toddler (no colon). The program crashes with an unhandled ValueError traceback.

```
(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler
Traceback (most recent call last):
  File "C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7\daycare_fees.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7\daycare_fees.py", line 17, in main
    result = parse_enrolment(user_input)
  File "C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7\daycare_fees.py", line 2, in parse_enrolment
    age_group, hours = text.split(':')
    ^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

4. Step 3: Hypothesis Fuzz Tests

4.1 Properties tested

- **Property 1, robustness:** no input of any kind (text, None, numbers, bytes, lists) may crash a function.
- **Property 2, correctness:** valid inputs give correct, in-range results (true mean, non-negative fee, discount never increases the fee).
- **Property 3, validation:** invalid inputs (missing or extra separators, non-numeric, nan, inf, negative, above 168 hours) are rejected with None, never turned into a number.

Fixed example cases (@example) and parametrized boundary cases are added next to the random Hypothesis inputs, so known bugs are always checked. The test file has 62 tests in total.

4.2 test_daycare_fees.py

```
"""Lab 7: Hypothesis fuzz tests for daycare_fees.py (all three functions).


Property 1: no input of any kind may crash a function.
Property 2: valid inputs give correct, in-range results.
Property 3: invalid inputs are rejected (None), never turned into a number.


Run:  pytest test_daycare_fees.py
"""
import pytest
from hypothesis import example, given, strategies as st


from daycare_fees import hours_average, parse_enrolment, payable_fee


MAX_WEEKLY_HOURS = 168


# Wider than the starter's text/None: also numbers, bytes and lists.
anything = st.one_of(
    st.text(), st.none(), st.integers(), st.floats(), st.binary(),
    st.lists(st.integers()),
)
letters = st.text(
    alphabet=st.characters(whitelist_categories=("Lu", "Ll")), min_size=1
)
hours_ok = st.integers(min_value=0, max_value=MAX_WEEKLY_HOURS)
money = st.floats(min_value=0, max_value=1e9, allow_nan=False)



# ---------------------------------------------- Property 1: never crash
@given(st.text() | st.none())
@example(None)
@example("toddler")
@example("a:b:c")
@example("toddler:abc")
def test_parse_enrolment_no_crash(s):
    try:
        parse_enrolment(s)
    except Exception as e:
        assert False, f"parse_enrolment crashed with: {e!r}"



@given(st.text() | st.none())
@example(None)
@example("")
@example("20,,30")
@example("abc")
def test_hours_average_no_crash(s):
    try:
        hours_average(s)
    except Exception as e:
        assert False, f"hours_average crashed with: {e!r}"



@given(st.text() | st.none())
@example(None)
@example("3000")
@example("abc|1")
```

```
@example("1|2|3")
def test_payable_fee_no_crash(s):
    try:
        payable_fee(s)
    except Exception as e:
        assert False, f"payable_fee crashed with: {e!r}"



@given(anything)
def test_all_functions_accept_any_type(value):
    for func in (parse_enrolment, hours_average, payable_fee):
        try:
            func(value)
        except Exception as e:
            assert False, f"{func.__name__} crashed on {value!r}: {e!r}"



# ---------------------------------------- Properties 2 and 3: parse_enrolment
@given(letters, st.floats(min_value=0, max_value=MAX_WEEKLY_HOURS))
def test_parse_enrolment_valid_roundtrip(group, hours):
    assert parse_enrolment(f"{group}:{hours}") == (group.strip().title(), hours)



@given(st.text())
def test_parse_enrolment_result_is_always_sane(s):
    result = parse_enrolment(s)
    if result is not None:
        group, hours = result
        assert group != ""
        assert 0 <= hours <= MAX_WEEKLY_HOURS



@given(st.text().filter(lambda s: ":" not in s))
def test_parse_enrolment_without_colon_is_rejected(s):
    assert parse_enrolment(s) is None



@given(letters, st.floats(min_value=0, max_value=MAX_WEEKLY_HOURS), st.text())
def test_parse_enrolment_extra_colon_is_rejected(group, hours, extra):
    assert parse_enrolment(f"{group}:{hours}:{extra}") is None



@pytest.mark.parametrize(
    "bad",
    ["", ":", ":25", "toddler:", "a:b:c", "toddler:abc", "toddler:nan",
     "toddler:inf", "toddler:1e999", "toddler:-5", "toddler:169",
     "toddler:25:30", "a:5:c", "::"],
)
def test_parse_enrolment_rejects_invalid(bad):
    assert parse_enrolment(bad) is None



def test_parse_enrolment_known_values():
    assert parse_enrolment("toddler:25") == ("Toddler", 25.0)
    assert parse_enrolment(" infant : 12.5 ") == ("Infant", 12.5)
    assert parse_enrolment("preschool:0") == ("Preschool", 0.0)
    assert parse_enrolment("preschool:168") == ("Preschool", 168.0)



# ---------------------------------------- Properties 2 and 3: hours_average
@given(st.lists(hours_ok, min_size=1))
def test_hours_average_is_true_mean_and_in_range(values):
    result = hours_average(",".join(map(str, values)))
    assert result == sum(values) / len(values)
    assert min(values) <= result <= max(values)



@given(st.lists(hours_ok, min_size=1), st.integers(min_value=-10**6, max_value=-1))
def test_hours_average_rejects_negative(values, negative):
    assert hours_average(",".join(map(str, values + [negative]))) is None



@given(st.lists(hours_ok, min_size=1), st.integers(min_value=MAX_WEEKLY_HOURS + 1))
```

```
def test_hours_average_rejects_over_168(values, too_big):
    assert hours_average(",".join(map(str, values + [too_big]))) is None



@given(st.text())
def test_hours_average_result_is_always_sane(s):
    result = hours_average(s)
    if result is not None:
        assert 0 <= result <= MAX_WEEKLY_HOURS



@pytest.mark.parametrize(
    "bad",
    ["", ",", "20,,30", "20,", ",20", "abc", "1e3", "2.5", "-5,10", "+5",
     "169", "1_0", "\u0663,\u0664"],
)
def test_hours_average_rejects_invalid(bad):
    assert hours_average(bad) is None



def test_hours_average_is_not_floored():
    assert hours_average("20,25") == 22.5
    assert hours_average("1,2") == 1.5
    assert hours_average("20,25,30") == 25.0
    assert hours_average(" 20 , 25 ") == 22.5



# ---------------------------------------- Properties 2 and 3: payable_fee
@given(money, money)
def test_payable_fee_valid_is_nonnegative_and_bounded(fee, discount):
    result = payable_fee(f"{fee}|{discount}")
    assert result is not None
    assert 0 <= result <= round(fee * 1.18, 2) + 0.01



@given(money, money, money)
def test_payable_fee_more_discount_never_costs_more(fee, d1, d2):
    low, high = sorted((d1, d2))
    assert payable_fee(f"{fee}|{high}") <= payable_fee(f"{fee}|{low}")



@given(st.floats(min_value=0.01, max_value=1e9), money)
def test_payable_fee_rejects_negative_fee(fee, discount):
    assert payable_fee(f"{-fee}|{discount}") is None



@given(money, st.floats(min_value=0.01, max_value=1e9))
def test_payable_fee_rejects_negative_discount(fee, discount):
    assert payable_fee(f"{fee}|{-discount}") is None



@given(st.text())
def test_payable_fee_result_is_always_sane(s):
    result = payable_fee(s)
    if result is not None:
        assert result == result and 0 <= result < float("inf")



@pytest.mark.parametrize(
    "bad",
    ["", "|", "3000", "1|2|3", "100|10|abc", "abc|1", "1|abc", "nan|0", "0|nan",
     "inf|0", "0|inf", "-100|0", "100|-50", "1e999|0", "1.7e308|0"],
)
def test_payable_fee_rejects_invalid(bad):
    assert payable_fee(bad) is None



def test_payable_fee_known_values():
    assert payable_fee("3000|250") == 3290.0
    assert payable_fee("100|1000") == 0.0     # discount larger than the bill
    assert payable_fee("0|0") == 0.0
    assert payable_fee(" 100 | 18 ") == 100.0
```

5. Step 4: Failing Tests on the Original Code

Command: python -m pytest test_daycare_fees.py (original daycare_fees.py in place)

Result: 57 of the 62 tests failed and 5 passed. Hypothesis reports a "Falsifying example" for each failing property, which is the smallest input it found that breaks the function.

```
FAILED test_daycare_fees.py::test_payable_fee_result_is_always_sane - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-ex
ceptions)
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[] - ValueError: not enough values to unpack (expected 2, got 1)
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[|] - ValueError: could not convert string to float: ''
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[3000] - ValueError: not enough values to unpack (expected 2, got 1)
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[1|2|3] - ValueError: too many values to unpack (expected 2)
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[100|10|abc] - ValueError: too many values to unpack (expected 2)
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[abc|1] - ValueError: could not convert string to float: 'abc'
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[1|abc] - ValueError: could not convert string to float: 'abc'
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[nan|0] - AssertionError: assert nan is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[0|nan] - AssertionError: assert nan is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[inf|0] - AssertionError: assert inf is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[0|inf] - AssertionError: assert 0 is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[-100|0] - AssertionError: assert 0 is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[100|-50] - AssertionError: assert 168.0 is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[1e999|0] - AssertionError: assert inf is None
FAILED test_daycare_fees.py::test_payable_fee_rejects_invalid[1.7e308|0] - AssertionError: assert inf is None
========================================== 57 failed, 5 passed in 6.99s ==========================================
```

```
=================================================== FAILURES ===================================================
_____________________________________ test_hours_average_is_true_mean_and_in_range _____________________________________

    @given(st.lists(hours_ok, min_size=1))
>   def test_hours_average_is_true_mean_and_in_range(values):
                ^^^

test_daycare_fees.py:118:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

values = [0, 1]

    @given(st.lists(hours_ok, min_size=1))
    def test_hours_average_is_true_mean_and_in_range(values):
        result = hours_average(",".join(map(str, values)))
>       assert result == sum(values) / len(values)
E       assert 0 == (1 / 2)
E        +  where 1 = sum([0, 1])
E        +  and   2 = len([0, 1])
E       Failing test case: test_hours_average_is_true_mean_and_in_range(
E           values=[0, 1],
E       )

test_daycare_fees.py:120: AssertionError
=============================================== short test summary info ================================================
FAILED test_daycare_fees.py::test_hours_average_is_true_mean_and_in_range - assert 0 == (1 / 2)
========================================== 1 failed, 61 deselected in 0.62s ==========================================
```

5.1 Bugs found

Function	Failing input	Original behaviour	Problem
parse_enrolment	None	AttributeError	No type check
parse_enrolment	toddler, a:b:c, toddler:abc	ValueError	Unpacking and float() not guarded
parse_enrolment	:25	returns (“”, 25.0)	Empty age group accepted
parse_enrolment	toddler:nan, :inf, :-5	returns nan, inf, -5.0	Non-finite and negative hours accepted
hours_average	None, “”, 20,,30, abc	AttributeError / ValueError	No validation of the CSV
hours_average	20,25 and 1,2	22 and 1	Floor division // drops the decimals (true 22.5 and 1.5)
hours_average	-5,10, 169, non-ASCII digits	accepted	Negative, impossible and non-ASCII values accepted
payable_fee	None, 3000, abc|1	AttributeError / ValueError	No type or format check
payable_fee	nan|0, inf|0, 1.7e308|0	returns nan / inf	Non-finite results returned
payable_fee	-100|0	returns 0	Negative fee silently hidden

Function	Failing input	Original behaviour	Problem
payable_fee	100|-50	returns 168.0	Negative discount raises the fee

6. Step 5: Fixing the Application

6.1 Changes made

Change	Reason
Return None for any invalid input; never raise	One clear contract: callers get a safe value instead of a crash.
Isinstance(text, str) check in every function	Fixes the None / non-string crashes.
Exact separator count (len(parts) != 2)	Fixes unpacking crashes and rejects extra separators.
_to_float helper rejects nan and inf; try/except around float()	Fixes float() crashes and non-finite values.
Hours limited to 0-168 (MAX_WEEKLY_HOURS)	Rejects negative and impossible hours.
Non-empty age group required	Rejects ":25".
Hours_average: sum(values) / len(values), only ASCII digits	True mean instead of floor division; rejects signs, decimals, underscores and non-ASCII digits.
Payable_fee: reject negative fee or discount, check result is finite, round to 2 decimals	Removes the fee-raising negative discount, the hidden negative fee and nan / inf results.
Main() prints a clear message on invalid input	The demo no longer shows a traceback.

6.2 Fixed daycare_fees.py

```
"""Daycare fee helpers: enrolment parsing, weekly-hours average, payable fee.

Every function is defensive: any invalid input (wrong type, bad format,
negative, non-finite or out-of-range numbers) makes it return ``None``
instead of raising an exception or returning a nonsensical value.
"""
import math

GST_RATE = 1.18
MAX_WEEKLY_HOURS = 168  # hours in a week; larger values cannot be valid



def _to_float(text):
    """Return ``text`` as a finite float, or None if it is not one."""
    Try:
        number = float(text)
    except (TypeError, ValueError, OverflowError):
        return None
    return number if math.isfinite(number) else None



def parse_enrolment(text):
    """Parse ``"age_group:hours"`` into ``(Age_group, hours)``.

    Returns None unless the text has exactly one ':', a non-empty age group
    and an hours value that is a finite number between 0 and 168.
    """
    if not isinstance(text, str):
        return None
    parts = text.split(":")
    if len(parts) != 2:
        return None
    age_group = parts[0].strip().title()
    hours = _to_float(parts[1])
    if not age_group or hours is None or not 0 <= hours <= MAX_WEEKLY_HOURS:
        return None
```

```
    return age_group, hours



def hours_average(hours_csv):
    “””Return the mean of comma-separated whole weekly hours (e.g. “20,25”).

    Returns None for non-strings and for any element that is empty, not a
    plain non-negative whole number, or above 168.
    “””
    if not isinstance(hours_csv, str):
        return None
    values = []
    for item in hours_csv.split(“,”):
        item = item.strip()
        if not (item.isascii() and item.isdigit()):
            return None
        try:
            value = int(item)
        except ValueError:
            return None
        if value > MAX_WEEKLY_HOURS:
            return None
        values.append(value)
    return sum(values) / len(values)



def payable_fee(text):
    “””Parse ``”fee|discount”`` and return the payable fee in rupees.

    The fee gets 18% GST added, then the discount is subtracted. The result
    is never negative and is rounded to 2 decimals. Returns None if the text
    is malformed or if fee or discount is negative or not finite.
    “””
    if not isinstance(text, str):
        return None
    parts = text.split(“|”)
    if len(parts) != 2:
        return None
    fee = _to_float(parts[0])
    discount = _to_float(parts[1])
    if fee is None or discount is None or fee < 0 or discount < 0:
        return None
    total = fee * GST_RATE – discount
    if not math.isfinite(total):
        return None
    return round(max(total, 0.0), 2)



def main():
    “””Interactive demo; invalid input prints a message instead of crashing.”””
    Print(“==== Running parse_enrolment ====”)
    user_input = input(“Enter age group:hours (e.g. toddler:25): “)
    result = parse_enrolment(user_input)
    if result is None:
        print(“Invalid input. Expected age_group:hours, e.g. toddler:25”)
    else:
        print(“Age Group and Hours:”, result)

    print(“\n==== Running hours_average ====”)
    user_input = input(“Enter weekly hours as CSV (e.g. 20,25,30): “)
    result = hours_average(user_input)
    if result is None:
        print(“Invalid input. Expected whole hours (0-168), e.g. 20,25,30”)
    else:
        print(“Average Hours:”, result)

    print(“\n==== Running payable_fee ====”)
    user_input = input(“Enter fee|discount (e.g. 3000|250): “)
    result = payable_fee(user_input)
    if result is None:
        print(“Invalid input. Expected fee|discount, e.g. 3000|250”)
    else:
        print(“Payable Fee:”, result)



if  == "__ma__"“
```

```
ain(”
```

6.3 Execution of the corrected application

Command: python daycare_fees.py

Valid input (toddler:25, 20,25, 3000|250). The average is now 22.5.

```
(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler:25
Age Group and Hours: ('Toddler', 25.0)

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,25
Average Hours: 22.5

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 3000|250
Payable Fee: 3290.0
```

Invalid input (toddler, 20,,30, 100|-50). The program reports the problem instead of crashing.

```
(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler
Invalid input. Expected age_group:hours, e.g. toddler:25

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,,30
Invalid input. Expected whole hours (0–168), e.g. 20,25,30

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 100|-50
Invalid input. Expected fee|discount, e.g. 3000|250

(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>
```

7. Step 6: Final Test Run on the Fixed Code

Command: python -m pytest test_daycare_fees.py

Result: all 62 tests pass. The run was repeated several times because Hypothesis generates new random inputs each time, and it passed every time.

```
(venv) C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7>python -m pytest test_daycare_fees.py
===================== test session starts =====================
platform win32 -- Python 3.13.15, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Samiksha\L6_L7_48_daycare_fees\Lab 7
plugins: hypothesis-6.170.15
collected 62 items

test_daycare_fees.py ................................... [ 56%]
.........................                                [100%]

====================== 62 passed in 1.81s ======================
```